

GitLab CI/CD — FF-Partner Bridge

Dokumentace k souboru `.gitlab-ci.yml`.

Obsah

1. Přehled pipeline
2. Prerekvizity
3. Globální proměnné
4. Stage 1 — build-and-test
5. Stage 2 — docker-push
6. Stage 3 — deploy-production
7. Nastavení CI/CD variables v GitLab
8. Nastavení deploy serveru
9. Nastavení Protected Environments
10. Kdy se pipeline spouští
11. Správa tagů Docker image
12. Ruční nasazení bez pipeline
13. Řešení problémů

1. Přehled pipeline



Pipeline má tři stages. Každá stage čeká na úspěšné dokončení předchozí (s výjimkou `deploy-production`, který navíc vyžaduje manuální spuštění).

2. Prerekvizity

GitLab runner

Job `docker-push` používá **Docker-in-Docker (DinD)**. Runner musí mít v `config.toml`:

```
[[runners]]
  executor = "docker"
  [runners.docker]
    privileged = true
    volumes = ["/certs/client", "/cache"]
```

Bez `privileged = true` job selže s chybou `Cannot connect to the Docker daemon`.

Alternativa bez privileged: Pokud nelze runner spustit v `privileged` módu (např. sdílené GitLab.com runnery), lze `docker-push` job nahradit buildem pomocí Kaniko.

Deploy server (XTuning on-premise)

Na cílovém serveru musí být:

- Docker Engine + Docker Compose plugin
- Uživatel `deploy` s oprávněním spustit `docker pull` a `docker compose`
- Adresář `/opt/ff-partner-bridge/` s funkčním `docker-compose.yml`
- SSH server naslouchající na standardním portu

Container registry

Image se pushuje do `registry.cendelin.eu`. Registry musí být dostupná jak z GitLab runneru (push), tak z deploy serveru (pull).

3. Globální proměnné

Definovány na úrovni celého souboru — platí pro všechny jobs:

Proměnná	Hodnota	Popis
<code>DOCKER_IMAGE</code>	<code>registry.cendelin.eu/ff-partner-bridge</code>	Cílový repozitář v registry
<code>DOTNET_VERSION</code>	<code>8.0</code>	Referenční verze .NET (informativní)
<code>DOCKER_DRIVER</code>	<code>overlay2</code>	Storage driver pro DinD
<code>DOCKER_TLS_CERTDIR</code>	<code>/certs</code>	TLS certifikáty pro bezpečné DinD připojení
<code>NUGET_PACKAGES</code>	<code>\$CI_PROJECT_DIR/.nuget/packages</code>	Lokální NuGet cache — musí být v rámci projektu pro fungování GitLab cache

4. Stage 1 — build-and-test

Job: build-and-test

Image: mcr.microsoft.com/dotnet/sdk:8.0

Spouštění: MR event, push na main, push na develop

Co dělá

1. Obnoví NuGet závislosti (`dotnet restore`)
2. Sestaví solution v Release konfiguraci (`dotnet build --configuration Release`)
3. Spustí všechny testy a запиše výsledky ve formátu TRX (`dotnet test --logger trx`)

NuGet cache

```
cache:
  key:
    files:
      - "**/*.csproj"
    prefix: nuget
  paths:
    - .nuget/packages/
  policy: pull-push
```

Cache klíč se mění pouze tehdy, když se změní některý `.csproj` soubor. Dokud se závislosti nemění, první `dotnet restore` dané pipeline použije existující cache a přeskočí stahování balíčků ze sítě. `pull-push` policy zajišťuje, že cache je vždy aktuální po každém úspěšném buildu.

Artefakty

```
artifacts:
  when: always # uloží i při selhání testů
  name: "test-results-${CI_PIPELINE_IID}"
  paths:
    - TestResults/**/*.trx
  expire_in: 30 days
```

TRX soubory jsou dostupné ke stažení v GitLab UI (Job → Browse artifacts).

Ukládají se i při selhání testů (`when: always`), aby bylo možné analyzovat příčinu selhání.

Poznámka — GitLab Test Reports UI: GitLab nativně neparsuje formát TRX.

Pro zobrazení pass/fail přehledu přímo v MR je potřeba přidat NuGet balíček

`JunitXml.TestLogger` a změnit logger na `--logger "junit;LogFilePath=..."`.

Aktuální konfigurace TRX je dostatečná pro audit — jen bez vizualizace v UI.

Pravidla spouštění

Podmínka	Spuštění
Otevření nebo aktualizace Merge Request	Ano

Podmínka	Spuštění
Push na <code>main</code>	Ano
Push na <code>develop</code>	Ano
Push na libovolnou jinou větev	Ne

5. Stage 2 — docker-push

Job: `docker-push`

Image: `docker:27` + service `docker:27-dind`

Spouštění: pouze push na `main` (ne MR, ne develop)

Co dělá

1. Přihlásí se do container registry (`docker login`)
2. Sestaví Docker image z `Dockerfile` v kořeni repozitáře
3. Přidá labely `git-commit` a `build-id` pro zpětnou dohledatelnost
4. Pushne image s tagem odpovídajícím sekvenčnímu číslu pipeline (`$CI_PIPELINE_IID`)
5. Odhlásí se z registry (`docker logout`)

Tagování image

```
variables:
  IMAGE_TAG: "$CI_PIPELINE_IID"
```

`CI_PIPELINE_IID` je sekvenční číslo pipeline v rámci projektu (1, 2, 3, ...).

Je předvídatelné a čitelné — na rozdíl od `CI_PIPELINE_ID`, který je globálně unikátní a má velká čísla. Výsledný tag vypadá např. takto:

```
registry.cendelin.eu/ff-partner-bridge:42
```

Image tag je pak předán do `deploy-production` jobu přes `$CI_PIPELINE_IID` (obě stage sdílejí stejnou pipeline, tedy stejnou hodnotu).

DinD a TLS

`DOCKER_TLS_CERTDIR: "/certs"` zajišťuje, že komunikace mezi `docker` clientem a `docker:27-dind` daemonem probíhá přes TLS. Bez TLS by job byl zranitelný vůči útokům typu MITM uvnitř runneru.

Přihlášení do registry

```
before_script:
  - echo "$REGISTRY_PASSWORD"
    | docker login registry.cendelin.eu
    --username "$REGISTRY_USERNAME"
    --password-stdin
```

Heslo se čte ze stdin, nikoli jako argument příkazové řádky. To zabrání jeho výskytu ve výpisu procesů (`ps aux`).

6. Stage 3 — deploy-production

Job: `deploy-production`

Image: `alpine:3.20`

Spouštění: push na `main`, **vyžaduje manuální spuštění** (`when: manual`)

Co dělá

1. Nainstaluje `openssh-client` a `curl` (Alpine nemá předinstalováno)
2. Zapiše SSH klíč a `known_hosts` ze CI/CD variables
3. Validuje `DEPLOY_HOST` (povoleny pouze `[a-zA-Z0-9._-]`) — ochrana před command injection
4. Přes SSH na deploy serveru:
 - Stáhne novou verzi image z registry (`docker pull`)
 - Spustí `docker compose up -d --no-build` s explicitním tagem
5. Provede health check — 10 pokusů s 5sekundovým intervalem (celkem 50 sekund)
6. Odstraní SSH klíč a `known_hosts` (`after_script: always`)

Health check

```
for i in $(seq 1 10); do
  ssh deploy@$DEPLOY_HOST \
    "curl --silent --fail 'http://localhost:8080/health' | grep -q 'healthy'"
  sleep 5
done
```

Health check probíhá **přes SSH na deploy serveru**, protože Bridge naslouchá pouze na interní síti XTuning (`172.24.0.1:8080`) — není přístupný přímo z GitLab runneru. Endpoint `/health` neobsahuje autentizaci a nevrací citlivá data.

Manuální gate

```
rules:
  - if: '$CI_COMMIT_BRANCH == "main" && $CI_PIPELINE_SOURCE == "push"'
    when: manual
```

Job se zobrazí v GitLab pipeline UI jako čekající na spuštění. Kdokoli s oprávněním `Developer` nebo vyšším ho může spustit kliknutím na tlačítko ☐ (Play). Pro omezení na konkrétní osoby viz sekci Nastavení Protected Environments.

GIT_STRATEGY: `none`

```
GIT_STRATEGY: none
```

Job neprovádí checkout repozitáře — nepotřebuje žádné zdrojové soubory.

Výsledkem je rychlejší start jobu a menší zátěž GitLab runneru.

Bezpečnost SSH klíče

```
after_script:
  - rm -f ~/.ssh/deploy_key ~/.ssh/known_hosts
```

SSH klíč je vždy smazán, i pokud předchází kroky selžou (`after_script` běží vždy). Tím se zabraňuje zůstání klíče v souborovém systému runneru při případném selhání nebo timeoutu.

7. Nastavení CI/CD variables v GitLab

Přejdi do **GitLab** → **projekt** → **Settings** → **CI/CD** → **Variables** a přidej:

Variable	Typ	Masked	Protected	Popis
REGISTRY_USERNAME	Variable	Ano	Doporučeno	Uživatelské jméno pro registry.cendelin.eu
REGISTRY_PASSWORD	Variable	Ano	Doporučeno	Heslo nebo token pro registry
DEPLOY_SSH_KEY	Variable	Ano	Doporučeno	Celý obsah privátního SSH klíče (včetně -----BEGIN a -----END řádků)
DEPLOY_KNOWN_HOSTS	Variable	Ano	Doporučeno	Viz postup níže
DEPLOY_HOST	Variable	Ano	Doporučeno	Hostname nebo IP deploy serveru
BRIDGE_HEALTH_URL	Variable	Ne	Ne	Výchozí: http://localhost:8080/health (volitelné)

Protected variables jsou dostupné pouze v *protected branches* a *tags* (typicky `main`). Doporučujeme toto nastavit, aby se *deploy credentials* nepoužily v *feature branches*.

Jak získat DEPLOY_KNOWN_HOSTS

Spusť na svém lokálním stroji nebo na libovolném trusted stroji:

```
ssh-keyscan -H <deploy-hostname-nebo-IP>
```

Výstup zkopíruj celý (včetně komentářů `#`) jako hodnotu proměnné `DEPLOY_KNOWN_HOSTS`.

Jak vygenerovat SSH klíčový pár pro deploy

```
# Vygeneruj klíč bez hesla (CI/CD nemůže zadávat hesla interaktivně)
ssh-keygen -t ed25519 -C "gitlab-cicd-ff-partner-bridge" -N "" -f ./deploy_key

# Obsah privátního klíče → GitLab CI/CD variable DEPLOY_SSH_KEY
cat ./deploy_key

# Obsah veřejného klíče → přidat na deploy serveru
cat ./deploy_key.pub
```

Na deploy serveru přidej veřejný klíč do `~deploy/.ssh/authorized_keys`:

```
echo "<obsah deploy_key.pub>" >> /home/deploy/.ssh/authorized_keys
chmod 600 /home/deploy/.ssh/authorized_keys
```

8. Nastavení deploy serveru

Deploy server (XTuning on-premise) musí splňovat následující:

Uživatel deploy

```
# Vytvořit uživatele deploy (pokud neexistuje)
useradd --system --shell /bin/bash --create-home deploy

# Přidat do skupiny docker (aby mohl spouštět docker příkazy bez sudo)
usermod -aG docker deploy
```

Adresářová struktura

```
/opt/ff-partner-bridge/
├─ docker-compose.yml      # produkční compose konfigurace
├─ secrets/                # Docker secrets (viz docker-compose.yml)
│   ├── azure_sql_conn.txt
│   ├── partner_cz_conn.txt
│   └─ ...
└─ bridge_admin_api_key.txt
```

Omezení oprávnění deploy uživatele (volitelně)

Pokud nechceš dávat deploy uživateli plný přístup k Dockeru, lze použít

`sudoers` pravidlo pro konkrétní příkazy:

```
# /etc/sudoers.d/deploy-bridge
deploy ALL=(ALL) NOPASSWD: /usr/bin/docker pull registry.cendelin.eu/ff-partner-bridge:*, \
    /usr/bin/docker compose -f /opt/ff-partner-bridge/docker-compose.yml up *
```

9. Nastavení Protected Environments

`when: manual` v pipeline umožňuje spustit deploy komukoli s přístupem `Developer+`.

Pro omezení na konkrétní osoby nebo skupiny:

1. Přejdi do **GitLab** → **projekt** → **Settings** → **CI/CD** → **Protected environments**
2. Klikni na **Protect an environment**
3. Vyber environment `production`
4. V sekci **Allowed to deploy** zvol konkrétní uživatele nebo skupiny
5. Ulož

Po nastavení uvidíš tlačítko ☐ v pipeline pouze vybraní uživatelé.

Ostatní vidí job jako `manual`, ale nemohou ho spustit.

10. Kdy se pipeline spouští

Událost	build-and-test	docker-push	deploy-production
Push na main	Ano	Ano (auto)	Ano (manual gate)
Push na develop	Ano	Ne	Ne
Push na jinou větev	Ne	Ne	Ne
Otevření Merge Request	Ano	Ne	Ne
Aktualizace Merge Request	Ano	Ne	Ne
Ruční spuštění pipeline (main)	Ano	Ano (auto)	Ano (manual gate)

11. Správa tagů Docker image

Formát tagu

Každý build na `main` vytvoří image s tagem odpovídajícím `$CI_PIPELINE_IID`

— sekvenčnímu číslu pipeline v rámci projektu:

```
registry.cendelin.eu/ff-partner-bridge:1
registry.cendelin.eu/ff-partner-bridge:2
registry.cendelin.eu/ff-partner-bridge:42
```

Deploy krok vždy nasadí konkrétní tag — nikdy `latest`. To zajišťuje deterministické chování a umožňuje rollback na libovolnou předchozí verzi.

Rollback na předchozí verzi

Pokud potřebuješ vrátit se na starší verzi bez nového commitu, přihlas se na deploy server a spusť:

```
cd /opt/ff-partner-bridge
IMAGE_TAG=<číslo-starší-pipeline> docker compose up -d --no-build ff-partner-bridge
```

Číslo pipeline najdeš v GitLab UI v historii pipeline nebo v Docker image labelu:


```
docker inspect registry.cendelin.eu/ff-partner-bridge:<tag> \
--format '{{index .Config.Labels "build-id"}}'
```

Čištění starých image

Na deploy serveru se starší tagy neklestou automaticky. Doporučujeme nastavit pravidelné čištění:

```
# Smazat image starší než 30 dní (ponechat posledních 5)
docker image prune --filter "until=720h" -f
```

12. Ruční nasazení bez pipeline

V případě výpadku GitLab nebo nutnosti hotfix deploye:

```
ssh deploy@<deploy-host>
cd /opt/ff-partner-bridge

# Přihlásit se do registry (jednorázově)
docker login registry.cendelin.eu

# Stáhnout konkrétní verzi
docker pull registry.cendelin.eu/ff-partner-bridge:<tag>

# Nasadit
IMAGE_TAG=<tag> docker compose up -d --no-build ff-partner-bridge

# Ověřit zdraví
curl http://localhost:8080/health
```

13. Řešení problémů

build-and-test selže na `dotnet restore`

Příznaky: Unable to load the service index for source https://api.nuget.org/v3/index.json

Řešení: Runner nemá přístup k internetu nebo NuGet.org. Nastav privátní NuGet feed v `nuget.config` v kořeni repozitáře nebo povolil odchozí HTTPS z runneru.

docker-push selže s `Cannot connect to the Docker daemon`

Příznaky: error during connect: Get http://docker:2376/v1.xx/info

Řešení: Runner nemá `privileged = true` v `config.toml`. Viz sekci

Prerekvizity.

deploy-production selže na SSH

Příznaky: Host key verification failed nebo Permission denied (publickey)

Řešení (Host key):

```
# Znovu vygeneruj DEPLOY_KNOWN_HOSTS
ssh-keyscan -H <deploy-host>
# Výstup ulož do GitLab CI/CD variable DEPLOY_KNOWN_HOSTS
```

Řešení (Permission denied):

```
# Na deploy serveru ověř authorized_keys
cat /home/deploy/.ssh/authorized_keys
# Musí obsahovat odpovídající veřejný klíč k DEPLOY_SSH_KEY
```

Health check selže po deployi

Příznaky: Bridge health check selhal po 50 sekundách

Příčiny a řešení:

1. Bridge se nespustí — ověř logy:

```
```bash
docker logs ff-partner-bridge --tail 50
```
```

2. Chybí secrets — ověř existenci souborů v `./secrets/`:

```
```bash
ls -la /opt/ff-partner-bridge/secrets/
```
```

3. Špatný `BRIDGE_HEALTH_URL` — výchozí URL je `http://localhost:8080/health`.

Pokud Bridge naslouchá na jiném portu, nastav `BRIDGE_HEALTH_URL` v CI/CD variables.

4. Pomalý start — Bridge potřebuje více než 50 sekund pro inicializaci

(typicky při prvním spuštění se studenými connection pooly). Lze upravit počet pokusů nebo interval přímo v pipeline, případně přidat `start_period` do `HEALTHCHECK` v `docker-compose.yml`.

deploy-production se nezobrazuje jako spustitelný

Příznaky: Uživatel nevidí tlačítko ☐ u `deploy-production` jobu.

Řešení: Job vyžaduje roli `Developer` nebo vyšší v projektu. Zkontroluj nastavení Protected Environments — viz sekci Nastavení Protected Environments.